

linkify.ink — Project Plan

High-Level Description

linkify.ink is a file-sharing tool where the “upload” never leaves the browser. Users add text files and small images, and the app compresses and encodes everything into the URL hash (#). Sharing a file means sharing a URL — there is no server, no database, no account. Anyone with the link can decode and download the files, or preview them in a sandboxed iframe.

The project has a strong archival ethos: the vendored library bundle is committed to git, there is no build step to run the app, and a working clone plus any static file server is all that is needed to use it indefinitely.

Features

Core

- Add multiple files (text or image) via drag-and-drop or file picker
- Edit text files in a built-in code editor panel
- Encode all files into the URL hash as a zstd-compressed, base64url-encoded tar
- Decode a hash URL and present the files for download or preview
- Preview HTML files (with working relative imports of JS/CSS/images) in a sandboxed service-worker-backed iframe on a random UUID subdomain
(`https://<uuid>.sandbox.linkify.ink`)
- Preview falls back gracefully (download only) when not on the hosted origin

Sharing & Encryption

- Share unencrypted (default)
- Share encrypted with a password (AES-256-GCM, key derived via Argon2id)
- Share encrypted with an ephemeral ECDH public key (for the `/receive` flow)

- Share button opens a modal to choose encryption mode and generate the final URL

`/receive` Flow

- Receiver opens `/receive`, which generates an ephemeral ECDH keypair in-browser
- Public key is displayed for copying and sending to the sender
- Receiver pastes the encrypted payload sent by the sender
- Files are decrypted locally and presented for download/preview

Image Handling

- All files go through zstd compression uniformly
- Optional lossy recompression of images to WebP (or AVIF where `canvas.toBlob` supports it) before tar + compress
- File is renamed to `.webp` / `.avif` on recompression; original format is not preserved (opt-in transform, user is informed)

UI

- Three-panel desktop layout: collapsible file sidebar | code editor | preview
- On mobile: stacked vertically (file list → editor → preview)
- Logo top-left (links to `/about`), Share button top-right
- Draggable divider between editor and preview panels
- Image files show an image viewer instead of the code editor
- `/about` page explaining the project and its archival nature

Implementation Details & Technology Choices

Runtime — no build step

The app is plain HTML + ES modules. `index.html` imports `app.js` directly. `app.js` and other source modules import from `libraries.bundle.js`. No compilation, transpilation, or bundler is needed to run the app.

Vendored libraries (`libraries.bundle.js`)

A single `libraries.js` re-exports everything needed from npm packages. `esbuild` (devDependency) bundles it to `libraries.bundle.js` (ESM, committed to git). The app never fetches from a CDN at runtime.

Rebuild command (contributors only):

```
npm run bundle-libs
# i.e.: esbuild libraries.js --bundle --format=esm --outfile=libraries.bundle.js
```

package.json devDependencies (contributors only — not needed to run the app):

```
{
  "devDependencies": {
    "typescript": "^5.x",
    "esbuild": "^0.25.x",
    "zstd-wasm": "^0.0.x",
    "argon2-browser": "^1.18.x",
    "marked": "^15.x",
    "preact": "^10.x",
    "htm": "^3.x"
  }
}
```

libraries.js (entrypoint for esbuild):

```
export { h, render, Fragment } from "preact"
export { useState, useEffect, useRef, useCallback } from "preact/hooks"
export { default as htm } from "htm"
export { marked } from "marked"
export * as ZstdWasm from "zstd-wasm"
export * as Argon2 from "argon2-browser"
```

Type checking

JSDoc comments on all public functions. `.d.ts` files for anything too complex for JSDoc.

`tsc --noEmit --allowJs --checkJs` for checking — no emit, no compilation.

UI framework

Preact + HTM — component model with no JSX compilation step.

```
import { h, render } from './libraries.bundle.js'
import { useState } from './libraries.bundle.js'
import htm from './libraries.bundle.js'
```

```
const html = htm.bind(h)
```

Routing

Hash-based routing handled manually (no router library). `window.location.pathname` determines the view: `/`, `/receive`, `/about`. The URL hash is always the payload, never used for routing.

URL / binary format

All content is base64url-encoded after processing. The binary layout before encoding:

Unencrypted:

```
[1 byte: version+encryption flags]
[2 bytes: metadata length (big-endian)]
[N bytes: metadata JSON (UTF-8)]
[zstd-compressed tar]
```

Password-encrypted:

```
[1 byte: version+encryption flags]
[16 bytes: Argon2id salt]
[12 bytes: AES-GCM IV]
-- ciphertext start --
[2 bytes: metadata length]
[N bytes: metadata JSON]
[zstd-compressed tar]
-- ciphertext end --
[16 bytes: AES-GCM auth tag]
```

ECDH-encrypted:

```
[1 byte: version+encryption flags]
[32 bytes: sender ephemeral P-256 public key (raw)]
[12 bytes: AES-GCM IV]
-- ciphertext start --
[2 bytes: metadata length]
[N bytes: metadata JSON]
[zstd-compressed tar]
-- ciphertext end --
[16 bytes: AES-GCM auth tag]
```

Version byte layout:

[2 bits: encryption type | 6 bits: format version]

Encryption types: 00 = none, 01 = password/Argon2id, 10 = ECDH, 11 = reserved

Metadata JSON example:

```
{ "preview": "index.html", "fullscreen": true }
```

Empty metadata → [0x00, 0x00] (zero-length, no JSON bytes).

Tar format

Files are packed into a standard POSIX ustar tar. A minimal tar implementation is hand-rolled (no library) — only the subset needed: regular files, 512-byte headers, filename + size + checksum. No symlinks, no permissions beyond defaults.

Compression

`zstd-wasm` at level 9 (good ratio / speed tradeoff for in-browser use). Wasm binary must be inlined into the bundle (base64-embedded) — verify esbuild does this, or find a pre-embedded build. This ensures `file://` loading works for decode (no fetch needed), though the preview feature itself still requires a server.

Encryption — WebCrypto (no library)

AES-256-GCM and ECDH/HKDF use the native `window.crypto.subtle` API exclusively. No crypto library is needed. Argon2id (for password KDF) uses `argon2-browser` (wasm), since WebCrypto does not include Argon2.

Image recompression

`canvas.toBlob(blob => ..., 'image/webp', 0.85)` for WebP. Feature-detect AVIF support at runtime and offer it if available. Recompression is opt-in per file. The file is renamed in the tar.

Preview sandbox

- Desktop layout iframes into `https://<uuid>.sandbox.linkify.ink/`
- That subdomain is served by a Cloudflare Worker that always returns the same small loader HTML regardless of subdomain
- The loader HTML registers a service worker scoped to its UUID origin
- Parent posts file contents via `postMessage` (checks `event.origin` strictly)

- Service worker intercepts fetch requests and serves files from the in-memory map
- `<iframe sandbox="allow-scripts allow-same-origin">` — **no** `allow-top-navigation`, `no allow-forms`, **no** `allow-popups`

Hosting

- Static files: Cloudflare Pages (or any static host)
- Wildcard subdomain: `*.sandbox.linkify.ink` → Cloudflare Worker (returns loader HTML)

File Structure

```
linkify.ink/
├─ index.html           # App shell, imports app.js
├─ app.js               # Root Preact component, routing
├─ components/
│   ├─ Editor.js        # Code editor panel
│   ├─ Preview.js       # Preview panel + iframe management
│   ├─ FileList.js      # Sidebar file list
│   ├─ ShareModal.js    # Encryption chooser + URL output
│   └─ ReceivePage.js   # /receive flow
├─ lib/
│   ├─ tar.js           # Hand-rolled minimal tar read/write
│   ├─ codec.js         # encode() / decode() — the full pipeline
│   ├─ crypto.js        # AES-GCM, ECDH, HKDF via WebCrypto
│   ├─ compress.js      # zstd-wasm wrappers (init + compress/decompress)
│   └─ base64url.js     # base64url encode/decode (no library)
├─ libraries.js         # esbuild entrypoint (re-exports npm deps)
├─ libraries.bundle.js  # Committed vendored bundle (do not edit manually)
├─ sandbox-loader.html  # Served by Cloudflare Worker on *.sandbox domain
├─ tsconfig.json        # noEmit, allowJs, checkJs — type checking only
└─ package.json         # devDependencies only (esbuild, typescript, npm deps)
```

Reference UI Mockup

The following is a reference HTML snippet for the three-panel layout. Use it as a starting point for `index.html` + `app.js`. The actual implementation should be Preact components.

```
<!-- App shell structure (conceptual — implement as Preact components) -->
```

```

<div class="app">

  <!-- Top bar -->
  <div class="topbar">
    <a class="logo" href="/about">
      <div class="logo-dot"></div>
      linkify.ink
    </a>
    <button class="share-btn" id="share">
      <i class="ti ti-link"></i> Share
    </button>
  </div>

  <!-- Main three-panel grid -->
  <div class="main">

    <!-- Left: collapsible file sidebar -->
    <aside class="sidebar">
      <div class="sidebar-header">
        Files
        <button aria-label="collapse sidebar">
          <i class="ti ti-layout-sidebar-left-collapse"></i>
        </button>
      </div>
      <ul class="file-list" role="list">
        <li class="file-item active">
          <i class="ti ti-file-type-html"></i> index.html
        </li>
        <li class="file-item">
          <i class="ti ti-file-type-js"></i> script.js
        </li>
      </ul>
      <div class="add-file">
        <button><i class="ti ti-plus"></i> add file</button>
      </div>
    </aside>

    <!-- Right: editor + preview split -->
    <div class="right">

      <!-- Editor panel -->
      <div class="panel editor-panel">
        <div class="panel-header">
          <i class="ti ti-code"></i> index.html
        </div>
        <div class="editor-body" contenteditable="true" spellcheck="false">
          <!-- file contents rendered here -->

```

```

        </div>
    </div>

    <!-- Drag handle -->
    <div class="divider-handle" role="separator" aria-orientation="horizontal">

    <!-- Preview panel -->
    <div class="panel preview-panel">
        <div class="panel-header">
            <i class="ti ti-eye"></i> preview - index.html
        </div>
        <iframe
            class="preview-iframe"
            sandbox="allow-scripts allow-same-origin"
            title="file preview">
        </iframe>
    </div>

    </div>
</div>
</div>

```

CSS grid layout (desktop):

```

.app {
    display: grid;
    grid-template-rows: 40px 1fr;
    height: 100dvh;
}

.main {
    display: grid;
    grid-template-columns: 200px 1fr;
    overflow: hidden;
}

.right {
    display: grid;
    grid-template-rows: 1fr 4px 1fr; /* editor / handle / preview */
    overflow: hidden;
}

/* Mobile */
@media (max-width: 640px) {
    .main {
        grid-template-columns: 1fr;
        grid-template-rows: auto 1fr 1fr;
    }
}

```

```
.sidebar {
  max-height: 200px;
  overflow-y: auto;
  border-right: none;
  border-bottom: 0.5px solid var(--border);
}
}
```

Task Breakdown

Phase 1 — Tooling & Vendoring

Task 1.1 — Scaffold the repo

- Create `package.json` with `devDependencies`: `typescript`, `esbuild`, `zstd-wasm`, `argon2-browser`, `marked`, `preact`, `htm`
- Create `tsconfig.json`:

```
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": true,
    "noEmit": true,
    "strict": true,
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ESNext", "DOM"]
  },
  "include": ["**/*.js"],
  "exclude": ["libraries.bundle.js", "node_modules"]
}
```

- Create `.gitignore`: `node_modules/`
- Add npm scripts:

```
"bundle-libs": "esbuild libraries.js --bundle --format=esm --outfile=libraries"
"typecheck": "tsc --noEmit"
```

Task 1.2 — Create `libraries.js` and build the bundle

- Write `libraries.js` re-exporting from all deps (see Implementation Details above)
- Run `npm install` then `npm run bundle-libs`
- Verify `libraries.bundle.js` is produced and includes all exports
- Investigate whether `zstd-wasm` and `argon2-browser` `wasm` binaries are inlined or emitted as sidecar files. If sidecar files are emitted, find or configure an embedded-`wasm` build of each. Document findings in a `VENDORING.md`.
- Commit `libraries.bundle.js` and `VENDORING.md`

Task 1.3 — Create `sandbox-loader.html`

- Small self-contained HTML page (no external dependencies)
- On load: register a service worker (`sandbox-sw.js`)
- Listen for a `postMessage` from the parent containing `{ files: { [name]: Uint8Array } }`
- Validate `event.origin` against the expected parent origin (hardcoded or configurable)
- Forward file map to service worker via its own `postMessage`
- Once SW is active and files are loaded, navigate the `iframe` to the entry file (e.g. `index.html`) within the SW scope

Task 1.4 — Create `sandbox-sw.js`

- Service worker that stores the file map in memory
 - Intercepts `fetch` events for same-origin requests
 - Serves matching files with correct MIME type inferred from extension
 - Returns 404 for unknown paths
-

Phase 2 — UI & Routing

Task 2.1 — `index.html` shell

- Minimal HTML boilerplate: `charset`, `viewport`, `title`
- Link to a `style.css` for global styles and CSS variables

- Single `<div id="app"></div>` mount point
- `<script type="module" src="app.js"></script>`
- No other scripts or stylesheets

Task 2.2 — Global styles (`style.css`)

- CSS custom properties for colors, spacing, border-radius
- Support light and dark mode via `prefers-color-scheme`
- Base reset (box-sizing, margin, padding)
- `.app`, `.topbar`, `.main`, `.sidebar`, `.right`, `.panel` layout styles as described in the reference mockup above
- Mobile responsive styles (stacked layout at $\leq 640\text{px}$)
- Styles for file list items, panel headers, divider handle, share button, logo

Task 2.3 — Root `app.js` and routing

- Import Preact + HTM from `libraries.bundle.js`
- Implement simple pathname router:
 - `/` → `EditorPage`
 - `/receive` → `ReceivePage`
 - `/about` → `AboutPage`
- On load, check `window.location.hash` — if non-empty, decode and load files (stubbed initially, wired up in Phase 4)
- Render top bar with logo and Share button into every page
- Mount root component to `#app`

Task 2.4 — `FileList.js` component

- Props: `files` (array of `{ name, type, content: Uint8Array }`), `activeFile`, `onSelect`, `onAdd`, `onDelete`
- Render collapsible sidebar with file list
- Collapse toggle hides the sidebar (CSS class `toggle` + narrow strip with expand button)
- "Add file" button opens native file picker (`<input type="file" multiple>`) and reads

files as `Uint8Array` via `FileReader`

- Right-click or trash icon on a file item to delete
- Show appropriate icon per file type (html, js, css, image, generic)

Task 2.5 — `Editor.js` component

- Props: `file ( { name, type, content: Uint8Array } ), onChange`
- For text files: render a plain `<textarea>` with monospace font, no syntax highlighting (CodeMirror can be considered later as a drop-in upgrade)
- Decode `content` (`Uint8Array` → UTF-8 string) for display; re-encode on change
- For image files: render an `<img>` with a blob URL, and offer the lossy recompression option (WebP/AVIF toggle + quality slider)
- Panel header shows the active filename

Task 2.6 — `Preview.js` component

- Props: `files, activeFile`
- For non-HTML text files: render a simple read-only formatted view (use `marked` for `.md` files, plain `<pre>` for others)
- For HTML files on the hosted origin: create a UUID, construct the sandbox iframe URL (`https://<uuid>.sandbox.linkify.ink/`), post files to it, display iframe
- For HTML files not on the hosted origin (local dev): display a notice that full preview requires the hosted version, but offer individual file download links
- For image files: display inline

Task 2.7 — `ShareModal.js` component

- Triggered by the Share button in the top bar
- Three-way radio: Unencrypted / Password / Public key
- Password mode: password input field (shown/hidden toggle)
- Public key mode: textarea to paste recipient's ECDH public key
- "Generate link" button → calls `encode()` from `codec.js` (stubbed in Phase 3, wired in Phase 4) → displays the URL with a copy button
- URL display shows character count and a rough "should work in most browsers"

indicator (warn if > 32 000 chars)

Task 2.8 — `ReceivePage.js` component

- On mount: generate ephemeral ECDH keypair via WebCrypto, display public key
- Textarea for pasting the encrypted payload from the sender
- “Decrypt” button → calls `decode()` → loads files into editor state
- `beforeunload` warning: “Your decryption key will be lost if you close this tab”

Task 2.9 — `AboutPage.js` component

- Static content explaining what linkify.ink is
 - Explain the archival nature: no server, no account, link = file
 - Explain the security model for encrypted links
 - Link to the GitHub repo
-

Phase 3 — Core Library Modules

Task 3.1 — `lib/base64url.js`

- `encode(buffer: Uint8Array): string` — standard base64url, no padding
- `decode(str: string): Uint8Array`
- No library — ~20 lines using `btoa` / `atob` and character substitution
- JSDoc types. Unit-test manually in browser console.

Task 3.2 — `lib/tar.js`

- `pack(files: { name: string, data: Uint8Array }[]): Uint8Array` — produces a valid POSIX ustar tar byte array
- `unpack(buffer: Uint8Array): { name: string, data: Uint8Array }[]` — parses a tar and returns files
- 512-byte headers, filename + size + checksum only
- No symlinks, no directories, no extended headers
- ~100 lines. JSDoc types. Test with a known tar file.

Task 3.3 — `lib/compress.js`

- `init(): Promise<void>` — initialise the `zstd-wasm` module (call once on startup)
- `compress(data: Uint8Array, level?: number): Uint8Array` — default level 9
- `decompress(data: Uint8Array): Uint8Array`
- Wrap the `ZstdWasm` import from `libraries.bundle.js`
- Handle `wasm` init lifecycle (ensure `init` is called before first use; safe to call multiple times)

Task 3.4 — `lib/crypto.js`

- All crypto via `window.crypto.subtle` only (no library for AES/ECDH)
- `argon2Derive(password: string, salt: Uint8Array): Promise<CryptoKey>` — uses Argon2 from `libraries.bundle.js`, returns an AES-256-GCM `CryptoKey`
- `aesEncrypt(key: CryptoKey, iv: Uint8Array, data: Uint8Array): Promise<Uint8Array>`
- `aesDecrypt(key: CryptoKey, iv: Uint8Array, data: Uint8Array): Promise<Uint8Array>`
- `generateEcdhKeypair(): Promise<{ publicKey: CryptoKey, privateKey: CryptoKey }>`
- `ecdhDerive(privateKey: CryptoKey, publicKey: CryptoKey): Promise<CryptoKey>` — does ECDH + HKDF to produce an AES-256-GCM `CryptoKey`
- `exportPublicKey(key: CryptoKey): Promise<Uint8Array>` — raw 65-byte P-256
- `importPublicKey(raw: Uint8Array): Promise<CryptoKey>`
- JSDoc types throughout.

Task 3.5 — `lib/codec.js`

- `encode(files, options): Promise<string>`

```

options: {
  encryption: 'none' | 'password' | 'ecdh',
  password?: string,
  recipientPublicKey?: Uint8Array,
  metadata?: object
}

```

Pipeline: tar → zstd compress → (encrypt) → prepend header → base64url → return full URL string (`https://linkify.ink/#<payload>`)

- `decode(hash: string, options): Promise<{ files, metadata }>`

```
options: {  
  password?: string,  
  privateKey?: CryptoKey  
}
```

Pipeline: base64url decode → parse header byte → (decrypt) → zstd decompress → untar → return files + metadata

- Assemble and parse the binary format exactly as specified in Implementation Details
 - Return typed errors for wrong password, corrupt data, version mismatch
-

Phase 4 — Wiring & Integration

Task 4.1 — Wire codec into app

- On app load: if `window.location.hash` is non-empty, call `decode()` and populate editor state with the returned files
- If decryption type is `password`, show a password prompt modal before decoding
- If decryption type is `ecdh`, navigate to `/receive` with the payload pre-filled

Task 4.2 — Wire Share modal

- Connect `ShareModal` to real `encode()` call
- On “Generate link”, encode current editor files with chosen encryption settings
- Display the resulting URL
- Test round-trip: encode → copy URL → open URL → files appear

Task 4.3 — Wire image recompression

- In `Editor.js`, when user toggles WebP/AVIF recompression, call `canvas.toBlob()` on the image, replace `file.content` and `file.name` in state
- Feature-detect AVIF encoding support and hide the option if unavailable

Task 4.4 — Wire preview sandbox

- When active file is HTML and origin is `linkify.ink` : generate UUID, open `https://<uuid>.sandbox.linkify.ink/` in iframe, wait for SW ready message, post files
- Add `beforeunload` guard on the receive page

Task 4.5 — Cloudflare deployment

- `wrangler.toml` for a Cloudflare Worker on `*.sandbox.linkify.ink`
- Worker always returns `sandbox-loader.html` content with correct headers (`Content-Type: text/html` , `Cross-Origin-Opener-Policy: same-origin` for SW)
- Cloudflare Pages config for the main `linkify.ink` static site
- Test wildcard subdomain routing

Task 4.6 — Polish & edge cases

- Handle oversized payloads gracefully (warn before encoding if estimated URL will exceed ~32 000 chars)
- Handle corrupt / unrecognised hash on load (show an error state, not a crash)
- Keyboard shortcut for Share (e.g. `Cmd/Ctrl+Shift+S`)
- `beforeunload` warning when editor has unsaved/unshared changes
- Empty state on first load (prompt to add a file or drag one in)
- Accessible labels on all icon-only buttons
- Smoke-test on Firefox, Chrome, Safari